

Knox College
Department of Computer Science
CS 142: Program design and methodology
Spring, 2025

Final Exam

Print your name and Knox login *neatly* in the spaces provided below. Please print legibly.

Name:
Knox login:

This is an *open note* exam. You are permitted to use written notes, but not a computer. Students whose native language is not English may use a dictionary. *Nothing else is permitted.*

Do all the problems in this booklet. If you get stuck, it may be beneficial to skip that problem and come back to it.

If a problem is unclear to you, include a brief explanation of how you interpret it.

Show your work! You will not get partial credit if I cannot figure out how you got your answer.

Question	Points	Score
1	16	
2	10	
3	10	
4	10	
5	11	
6	9	
7	12	
8	12	
9	10	
Total	100	

1. **Running times (16 points).** Give the running time in Big-O notation for each of the code segments below. Assume that `A` is an array of integers and that `n = A.length`.

(a) `get` in an `ArrayList` containing `n` values

(b) `add` to a `LinkedList` containing `n` values

(c) `push` in an array-based implementation of a `Stack` containing `n` values

(d) `dequeue` in a linked-memory implementation of a `Queue` containing `n` values

(e) `enqueue` in a heap implementation of a `PriorityQueue` containing `n` values

(f)

```
ArrayList<Integer> list = new ArrayList<Integer>();
for(int i=0; i < A.length; i++)
    list.add(A[i]);
```

(g)

```
int num = 0;
for(int i=0; i < A.length; i++)
    if(A[i] == num)
        num++;
for(int j=0; j < num; j++)
    if(A[j] == A[A.length-1-j])
        num--;
```

(h)

```
TreeSet<Integer>[] set = new TreeSet<Integer>();
for(int i=0; i < A.length; i++) {
    set.add(A[i]);
}
```

2. **Sorting (10 points).** Consider the array of integers

7	3	1	6	5	2	8	4
---	---	---	---	---	---	---	---

Draw the sequence of swaps that result from sorting this array (into increasing order) using selection sort.

3. **Binary Search Trees (10 points).** Using the implementation of binary search tree operations we discussed in class, draw the tree that results from inserting 5, 8, 10, 7, 4, 6, 2, 1, 9, and 3 into an initially-empty tree (in that order). In order to ensure that you get partial credit, I suggest that you draw the tree after each of these insertions.

4. **Testing (10 points).** Consider a possible List method `set`, which takes an index and a value. It changes the value at the given index. (There must already be a value at that index or the method throws an `IndexOutOfBoundsException`.) Write test cases for this method in our `RecursiveList` class, which you may assume already implements `addFront`, `add` (adds to end), `size`, `get`, `contains`, and `remove` (which takes a value),

```
public class RecursiveListTest {
    RecursiveList<String> list;

    @Before
    public void setUp() throws Exception {
        list = new RecursiveList<String>();
    }

    @Test
    public void testSet() {
```

5. **Implementing recursive list method (11 points).** Write a recursive implementation for **set** (which takes an index and a value, updating the value at the given index or throwing an `IndexOutOfBoundsException`). In each part below, give the code for **set** in the corresponding interface or class.

(a) **set** in `Node<T>` interface:

(b) **set** in `EmptyNode<T>` class:

(c) **set** in `NonEmptyNode<T>` class:

(d) **set** in `RecursiveList<T>` class:

6. **fork-join (9 points).** Another reduction operation is creating a histogram. This is an array where the cell of each index contains the number of times that index appears in the input. For example, input 1, 0, 1, 3, 1 would yield the output array (1, 3, 0, 1) because there is one 0, three 1s, no 2s, and one 3. Below is part of a class to use with fork-join to create a histogram in parallel. Assume that the input `array` only contains the values 0–3 so the return value of `compute` is an array of length 4. Fill in the missing code to complete this class.

```
public class CountArray extends RecursiveTask<int[]>
    int[] array;    //input array
    int hi;         //bounds on part to count
    int low;

    public CountArray(int[] array, int low, int hi) {
        this.array = array;  this.low = low;  this.hi = hi;
    }

    protected int[] compute() {
        if(hi - low < 100) {
            //FILL IN BASE CASE

        }

        //FILL IN RECURSIVE CASE

    }

}
```

7. **Tree traversals (12 points).** Draw the unique binary tree that has an inorder traversal of 5, 3, 7, 11, 2, 8, 12, 10, 1, 6, 4, 9 and a postorder traversal of 5, 11, 7, 2, 3, 12, 1, 10, 4, 9, 6, 8. Note that this will not necessarily be a binary search tree.

8. **Radix sort (12 points).** Use radix sort (by digit) to sort the following numbers:

200, 321, 402, 13, 114, 231, 301, 220, 114, 340, 422, 433, 341, 114, 120, 34, 220, 314.

(For convenience, each digit is 0–4; you only have to include buckets for these digits.) Show the bucket contents and resulting list after each digit is considered. (So I want 3 figures and lists...) The numbers should start in the order they are given above. Note that you do not need to write any code for this problem.

9. **Design problems (10 points).** Describe how you would solve each of the following problems using data structures we've learned in this class. Specify the ADTs you would use, the implementation you would select if it matters, and how you would use them to efficiently support the required operations.

- (a) Modern computing systems sometimes have search capabilities that let you search based not only on the names of files, but also their contents. For example, I could look for all the documents (text files, Word documents, and webpages for example) that contain the phrase “data structures”. Given the size of modern computer memories, it is impractical for the computer to search for this phrase with a linear scan through its entire memory. Instead, it builds a data structure that tells it which documents contain each word before I make my query. Then, when I query a specific phrase, it uses this data structure to identify the documents containing each word in the phrase (“data” and “structures” in my example), greatly narrowing the search.

What ADTs would you use to find all the documents containing a single word?

- (b) During registration, students attempt to enroll in class, but the classes are sometimes full. The new registration system will support waiting lists. Thus, when a student attempts to add a class that is already full, they are added to the waiting list. When an enrolled student drops the class, the waiting list is consulted and the student with the highest class standing (seniors first, then juniors, then sophomores, and finally first years) will be enrolled. Among waiting list students with the same class standing, the one who attempted to add the class earliest is added. The system must support adding and dropping a class, plus “get status”, which returns whether the student is enrolled, on the waiting list, or neither for a particular class. What ADTs would you use to support these operations for a single class? (Your data structure will be replicated for all of Knox's classes.)